

zipCode2.0

Generated by Doxygen 1.13.2

1 ZipCode2.0	1
1.1 Zip Code Data Processor with Index Support	1
1.1.1 Description	1
1.1.2 Features	1
1.1.3 Files in the Project	1
1.1.4 Compilation Instructions	2
1.1.5 Running the Program	2
1.1.5.1 1. Interactive Mode	2
1.1.5.2 2. ZIP Code Lookup via Command Line	2
1.1.6 Output Example (ZIP Lookup)	2
1.1.7 Output Example (State Summary)	2
1.1.8 Error Handling	2
1.1.9 Authors	2
2 Class Index	3
2.1 Class List	3
3 File Index	5
3.1 File List	5
4 Class Documentation	7
4.1 CSVBuffer Class Reference	7
4.1.1 Detailed Description	8
4.1.2 Constructor & Destructor Documentation	8
4.1.2.1 CSVBuffer()	8
4.1.3 Member Function Documentation	9
4.1.3.1 buildPrimaryKeyIndex()	9
4.1.3.2 convertCSVToLengthIndicated()	9
4.1.3.3 generateStateTable()	10
4.1.3.4 getRecords()	11
4.1.3.5 loadLengthIndicatedRecords()	11
4.1.3.6 loadRecords()	11
4.1.3.7 searchByZipCodes()	12
4.1.4 Member Data Documentation	13
4.1.4.1 file	13
4.1.4.2 records	13
4.2 Field Struct Reference	14
4.2.1 Detailed Description	14
4.2.2 Member Data Documentation	14
4.2.2.1 format	14
4.2.2.2 isPrimaryKey	14
4.2.2.3 name	14
4.2.2.4 type	15

4.3 Header Class Reference	15
4.3.1 Detailed Description	15
4.3.2 Constructor & Destructor Documentation	16
4.3.2.1 Header() [1/2]	16
4.3.2.2 Header() [2/2]	16
4.3.3 Member Function Documentation	16
4.3.3.1 readFromFile()	16
4.3.3.2 writeToFile()	17
4.3.4 Member Data Documentation	17
4.3.4.1 fields	17
4.3.4.2 fieldsPerRecord	17
4.3.4.3 fileStructureType	17
4.3.4.4 primaryKey	17
4.3.4.5 primaryKeyFileName	18
4.3.4.6 recordCount	18
4.3.4.7 sizeFormatType	18
4.3.4.8 sizeInBytes	18
4.3.4.9 version	18
4.4 ZipRecord Struct Reference	18
4.4.1 Detailed Description	19
4.4.2 Member Data Documentation	19
4.4.2.1 county	19
4.4.2.2 latitude	19
4.4.2.3 longitude	19
4.4.2.4 placeName	20
4.4.2.5 state	20
4.4.2.6 zipCode	20
5 File Documentation	21
5.1 CSVBuffer.cpp File Reference	21
5.1.1 Detailed Description	21
5.2 CSVBuffer.cpp	22
5.3 CSVBuffer.h File Reference	25
5.4 CSVBuffer.h	25
5.5 HeaderBuffer.h File Reference	26
5.5.1 Detailed Description	27
5.6 HeaderBuffer.h	27
5.7 main.cpp File Reference	29
5.7.1 Detailed Description	29
5.7.2 Function Documentation	30
5.7.2.1 main()	30
5.8 main.cpp	31

5.9 README.md File Reference	32
Index	33

Chapter 1

ZipCode2.0

1.1 Zip Code Data Processor with Index Support

1.1.1 Description

This program processes U.S. ZIP code data from a CSV or length-indicated file and supports both:

- Generating a state-wise summary of extreme ZIP codes.
- Searching ZIP codes using a fast in-memory primary key index via command-line flags.

1.1.2 Features

- Reads ZIP code data from a standard CSV or length-indicated format.
- Converts CSV to a length-indicated file with a structured header.
- Extracts fields such as zip code, city, state, latitude, and longitude.
- Generates a summary of easternmost, westernmost, northernmost, and southernmost ZIP codes per state.
- Creates and uses a **primary key index** in RAM for fast ZIP code lookups.
- Accepts `-z#####` flags (e.g. `-z56301`) from the command line to lookup specific ZIP code records.
- Includes error handling for malformed input and missing files.

1.1.3 Files in the Project

- `main.cpp` – The main application logic, including command-line parsing and user interaction.
- `CSVBuffer.h` / `CSVBuffer.cpp` – Handles reading, indexing, converting, and summarizing ZIP code records.
- `HeaderBuffer.h` – Defines the structure and reading/writing of metadata headers for length-indicated files.
- `us_postal_codes.csv` – Sample CSV input file.
- `ZipCodes` – Sample converted length-indicated file (with header + records).
- `README.md` – This documentation.

1.1.4 Compilation Instructions

Use the following command to compile:

```
g++ -o myProgram main.cpp CSVBuffer.cpp
```

This creates an executable named `myProgram`.

1.1.5 Running the Program

1.1.5.1 1. Interactive Mode

Launch the program without flags:

```
./myProgram
```

You'll be prompted to:

- Convert a CSV file to length-indicated format (Option 1), **or**
- Process a file and generate a summary table (Option 2)

1.1.5.2 2. ZIP Code Lookup via Command Line

You can directly lookup one or more ZIP codes using `-z#####` flags:

```
./myProgram ZipCodes -z56301 -z90210 -z99999
```

If a ZIP code is found, the full record will be printed with all field labels.
If not, an appropriate message will be shown.

The file (`ZipCodes`) must be the converted length-indicated format.

1.1.6 Output Example (ZIP Lookup)

```
Zip Code: 56301, Place Name: Saint Cloud, State: MN, County: Stearns, Latitude: 45.555, Longitude: -94.167
Zip Code 99999 not found in the file.
```

1.1.7 Output Example (State Summary)

```
State, Easternmost, Westernmost, Northernmost, Southernmost
MN, 55001, 56763, 56701, 55044
TX, 78330, 79851, 79083, 78575
...
```

1.1.8 Error Handling

- If input or output files can't be opened, a descriptive error will be printed.
- Malformed rows are skipped gracefully.
- Invalid ZIP code flags will show a warning.

1.1.9 Authors

Cha Vue, Sofia Hoffman, Alexander Miller, Zoljargal Enkhbayar, Yohannes Niguesse, Fatha Abdi

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

CSVBuffer	Class to handle CSV data and extract structured zip code records	7
Field	Structure to represent a field in the header	14
Header	Class to manage header metadata for record-based file storage	15
ZipRecord	Structure to store zip code information	18

Chapter 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

CSVBuffer.cpp		
	Implementation of CSVBuffer class functions	21
CSVBuffer.h		25
HeaderBuffer.h		
	Defines a class for handling header records in data files	26
main.cpp		
	Main application to process Zip Code data from CSV or length-indicated file	29

Chapter 4

Class Documentation

4.1 CSVBuffer Class Reference

Class to handle CSV data and extract structured zip code records.

```
#include <CSVBuffer.h>
```

Public Member Functions

- [CSVBuffer](#) (const string &filename)
Constructor that opens a CSV or length-indicated file.
- const vector< [ZipRecord](#) > & [getRecords](#) () const
Retrieves all zip code records.
- void [generateStateTable](#) () const
Generates a state-wise summary of extreme zip codes.
- unordered_map< int, [ZipRecord](#) > [buildPrimaryKeyIndex](#) () const
Builds an in-memory primary key index of zip codes.
- void [searchByZipCodes](#) (const vector< int > &zipCodes) const
Searches and prints zip code records from a list.

Static Public Member Functions

- static void [convertCSVToLengthIndicated](#) (const string &inputFile, const string &outputFile)
Converts a standard CSV file to a length-indicated format.

Private Member Functions

- void [loadRecords](#) ()
Loads records from the CSV file into memory.
- void [loadLengthIndicatedRecords](#) ()
Loads length-indicated records from a file.

Private Attributes

- ifstream [file](#)
File stream to read CSV data.
- vector< [ZipRecord](#) > [records](#)
Vector to store all zip code records.

4.1.1 Detailed Description

Class to handle CSV data and extract structured zip code records.

Definition at line 29 of file [CSVBuffer.h](#).

4.1.2 Constructor & Destructor Documentation

4.1.2.1 CSVBuffer()

```
CSVBuffer::CSVBuffer (
    const string & filename)
```

Constructor that opens a CSV or length-indicated file.

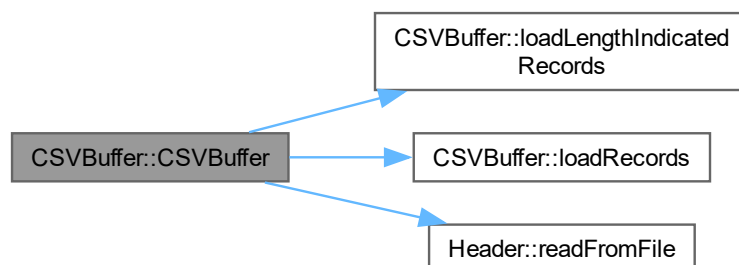
Constructs a [CSVBuffer](#) object and loads data from a given file.

Parameters

<i>filename</i>	Name of the input file.
<i>filename</i>	The name of the CSV or length-indicated file.

Definition at line 23 of file [CSVBuffer.cpp](#).

Here is the call graph for this function:



4.1.3 Member Function Documentation

4.1.3.1 buildPrimaryKeyIndex()

```
unordered_map< int, ZipRecord > CSVBuffer::buildPrimaryKeyIndex () const
```

Builds an in-memory primary key index of zip codes.

Builds an in-memory index mapping ZIP codes to records.

Returns

An unordered map from zip code to [ZipRecord](#).

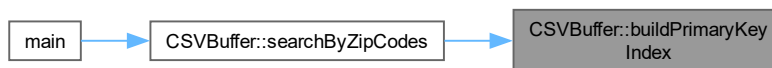
This function creates a hash table (`unordered_map`) that allows fast lookup of [ZipRecord](#) entries based on their zip code.

Returns

An unordered map of ZIP codes to corresponding [ZipRecord](#) objects.

Definition at line 44 of file [CSVBuffer.cpp](#).

Here is the caller graph for this function:



4.1.3.2 convertCSVToLengthIndicated()

```
void CSVBuffer::convertCSVToLengthIndicated (
    const string & inputFile,
    const string & outputFile) [static]
```

Converts a standard CSV file to a length-indicated format.

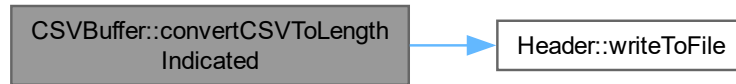
Converts a CSV file to a length-indicated format with header metadata.

Parameters

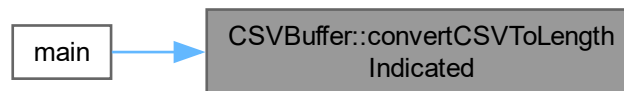
<i>inputFile</i>	The input CSV filename.
<i>outputFile</i>	The output filename in length-indicated format.
<i>inputFile</i>	The name of the input CSV file.
<i>outputFile</i>	The name of the output file.

Definition at line 142 of file [CSVBuffer.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.3.3 generateStateTable()

```
void CSVBuffer::generateStateTable () const
```

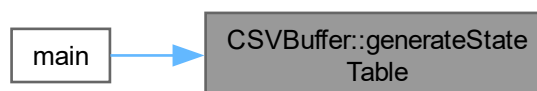
Generates a state-wise summary of extreme zip codes.

Generates a summary file of extreme ZIP codes (east, west, north, south) per state.

Finds and outputs the easternmost, westernmost, northernmost, and southernmost zip codes per state.

Definition at line 224 of file [CSVBuffer.cpp](#).

Here is the caller graph for this function:



4.1.3.4 getRecords()

```
const vector< ZipRecord > & CSVBuffer::getRecords () const
```

Retrieves all zip code records.

Returns

A constant reference to a vector containing all [ZipRecord](#) structures.

4.1.3.5 loadLengthIndicatedRecords()

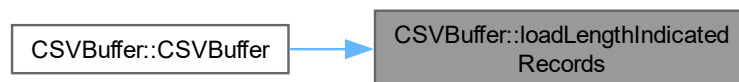
```
void CSVBuffer::loadLengthIndicatedRecords () [private]
```

Loads length-indicated records from a file.

Loads ZIP code records from a length-indicated file.

Definition at line 110 of file [CSVBuffer.cpp](#).

Here is the caller graph for this function:



4.1.3.6 loadRecords()

```
void CSVBuffer::loadRecords () [private]
```

Loads records from the CSV file into memory.

Loads ZIP code records from a standard CSV file.

Definition at line 83 of file [CSVBuffer.cpp](#).

Here is the caller graph for this function:



4.1.3.7 searchByZipCodes()

```
void CSVBuffer::searchByZipCodes (
    const vector< int > & zipCodes) const
```

Searches and prints zip code records from a list.

Searches and displays ZIP code records for each code in a given list.

Parameters

<i>zipCodes</i>	Vector of zip codes to look up.
<i>zipCodes</i>	A vector of ZIP codes to search for.

Definition at line 65 of file [CSVBuffer.cpp](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.1.4 Member Data Documentation

4.1.4.1 file

```
ifstream CSVBuffer::file [private]
```

File stream to read CSV data.

Definition at line 31 of file [CSVBuffer.h](#).

4.1.4.2 records

```
vector<ZipRecord> CSVBuffer::records [private]
```

Vector to store all zip code records.

Definition at line 32 of file [CSVBuffer.h](#).

The documentation for this class was generated from the following files:

- [CSVBuffer.h](#)
- [CSVBuffer.cpp](#)

4.2 Field Struct Reference

Structure to represent a field in the header.

```
#include <HeaderBuffer.h>
```

Public Attributes

- string [name](#)
Field name.
- string [type](#)
Data type of the field.
- string [format](#)
Format of the field data.
- bool [isPrimaryKey](#) = false
Indicates if this field is the primary key.

4.2.1 Detailed Description

Structure to represent a field in the header.

Definition at line 19 of file [HeaderBuffer.h](#).

4.2.2 Member Data Documentation

4.2.2.1 format

```
string Field::format
```

Format of the field data.

Definition at line 22 of file [HeaderBuffer.h](#).

4.2.2.2 isPrimaryKey

```
bool Field::isPrimaryKey = false
```

Indicates if this field is the primary key.

Definition at line 23 of file [HeaderBuffer.h](#).

4.2.2.3 name

```
string Field::name
```

[Field](#) name.

Definition at line 20 of file [HeaderBuffer.h](#).

4.2.2.4 type

```
string Field::type
```

Data type of the field.

Definition at line 21 of file [HeaderBuffer.h](#).

The documentation for this struct was generated from the following file:

- [HeaderBuffer.h](#)

4.3 Header Class Reference

Class to manage header metadata for record-based file storage.

```
#include <HeaderBuffer.h>
```

Public Member Functions

- [Header](#) ()
Default constructor initializing fields to default values.
- [Header](#) (const string &[fileStructureType](#), const string &[version](#), int [sizeInBytes](#), const string &[sizeFormatType](#), const string &[primaryKeyFileName](#), int [recordCount](#), const vector< [Field](#) > &[fields](#), const string &[primaryKey](#))
Parameterized constructor to initialize with custom values.
- void [writeToFile](#) (ofstream &out) const
Writes the header to a file.
- bool [readFromFile](#) (ifstream &in)
Reads the header from a file.

Public Attributes

- string [fileStructureType](#)
- string [version](#)
- int [sizeInBytes](#)
- string [sizeFormatType](#)
- string [primaryKeyFileName](#)
- int [recordCount](#)
- int [fieldsPerRecord](#)
- vector< [Field](#) > [fields](#)
- string [primaryKey](#)

4.3.1 Detailed Description

Class to manage header metadata for record-based file storage.

Definition at line 27 of file [HeaderBuffer.h](#).

4.3.2 Constructor & Destructor Documentation

4.3.2.1 Header() [1/2]

```
Header::Header () [inline]
```

Default constructor initializing fields to default values.

Definition at line 40 of file [HeaderBuffer.h](#).

4.3.2.2 Header() [2/2]

```
Header::Header (
    const string & fileStructureType,
    const string & version,
    int sizeInBytes,
    const string & sizeFormatType,
    const string & primaryKeyFileName,
    int recordCount,
    const vector< Field > & fields,
    const string & primaryKey) [inline]
```

Parameterized constructor to initialize with custom values.

Definition at line 45 of file [HeaderBuffer.h](#).

4.3.3 Member Function Documentation

4.3.3.1 readFromFile()

```
bool Header::readFromFile (
    ifstream & in) [inline]
```

Reads the header from a file.

Parameters

<i>in</i>	Input file stream to read the structured header.
-----------	--

Returns

True if successfully read, false otherwise.

Definition at line 77 of file [HeaderBuffer.h](#).

Here is the caller graph for this function:



4.3.3.2 writeToFile()

```
void Header::writeToFile (
    ostream & out) const [inline]
```

Writes the header to a file.

Parameters

<i>out</i>	Output file stream to write the structured header.
------------	--

Definition at line 58 of file [HeaderBuffer.h](#).

Here is the caller graph for this function:



4.3.4 Member Data Documentation

4.3.4.1 fields

```
vector<Field> Header::fields
```

Definition at line 36 of file [HeaderBuffer.h](#).

4.3.4.2 fieldsPerRecord

```
int Header::fieldsPerRecord
```

Definition at line 35 of file [HeaderBuffer.h](#).

4.3.4.3 fileStructureType

```
string Header::fileStructureType
```

Definition at line 29 of file [HeaderBuffer.h](#).

4.3.4.4 primaryKey

```
string Header::primaryKey
```

Definition at line 37 of file [HeaderBuffer.h](#).

4.3.4.5 primaryKeyFileName

```
string Header::primaryKeyFileName
```

Definition at line 33 of file [HeaderBuffer.h](#).

4.3.4.6 recordCount

```
int Header::recordCount
```

Definition at line 34 of file [HeaderBuffer.h](#).

4.3.4.7 sizeFormatType

```
string Header::sizeFormatType
```

Definition at line 32 of file [HeaderBuffer.h](#).

4.3.4.8 sizeInBytes

```
int Header::sizeInBytes
```

Definition at line 31 of file [HeaderBuffer.h](#).

4.3.4.9 version

```
string Header::version
```

Definition at line 30 of file [HeaderBuffer.h](#).

The documentation for this class was generated from the following file:

- [HeaderBuffer.h](#)

4.4 ZipRecord Struct Reference

Structure to store zip code information.

```
#include <CSVBuffer.h>
```

Public Attributes

- int [zipCode](#)
Zip code number.
- string [placeName](#)
Name of the place.
- string [state](#)
Two-letter state abbreviation.
- string [county](#)
County name.
- double [latitude](#)
Latitude coordinate.
- double [longitude](#)
Longitude coordinate.

4.4.1 Detailed Description

Structure to store zip code information.

Definition at line 19 of file [CSVBuffer.h](#).

4.4.2 Member Data Documentation

4.4.2.1 county

```
string ZipRecord::county
```

County name.

Definition at line 23 of file [CSVBuffer.h](#).

4.4.2.2 latitude

```
double ZipRecord::latitude
```

Latitude coordinate.

Definition at line 24 of file [CSVBuffer.h](#).

4.4.2.3 longitude

```
double ZipRecord::longitude
```

Longitude coordinate.

Definition at line 25 of file [CSVBuffer.h](#).

4.4.2.4 placeName

```
string ZipRecord::placeName
```

Name of the place.

Definition at line 21 of file [CSVBuffer.h](#).

4.4.2.5 state

```
string ZipRecord::state
```

Two-letter state abbreviation.

Definition at line 22 of file [CSVBuffer.h](#).

4.4.2.6 zipCode

```
int ZipRecord::zipCode
```

Zip code number.

Definition at line 20 of file [CSVBuffer.h](#).

The documentation for this struct was generated from the following file:

- [CSVBuffer.h](#)

Chapter 5

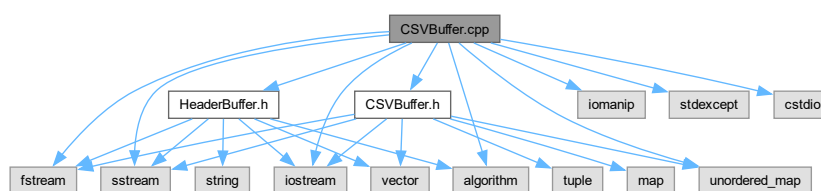
File Documentation

5.1 CSVBuffer.cpp File Reference

Implementation of [CSVBuffer](#) class functions.

```
#include "CSVBuffer.h"  
#include "HeaderBuffer.h"  
#include <iostream>  
#include <fstream>  
#include <sstream>  
#include <iomanip>  
#include <stdexcept>  
#include <algorithm>  
#include <unordered_map>  
#include <cstdio>
```

Include dependency graph for CSVBuffer.cpp:



5.1.1 Detailed Description

Implementation of [CSVBuffer](#) class functions.

This file provides functionality for reading CSV or length-indicated ZIP code data, building a primary key index, converting CSV to a structured format, and generating summaries.

Definition in file [CSVBuffer.cpp](#).

5.2 CSVBuffer.cpp

[Go to the documentation of this file.](#)

```

00001
00005
00006 #include "CSVBuffer.h"
00007 #include "HeaderBuffer.h"
00008 #include <iostream>
00009 #include <fstream>
00010 #include <sstream>
00011 #include <iomanip>
00012 #include <stdexcept>
00013 #include <algorithm>
00014 #include <unordered_map>
00015 #include <cstdio>
00016
00017 using namespace std;
00018
00023 CSVBuffer::CSVBuffer(const string& filename) {
00024     file.open(filename);
00025     if (!file.is_open()) {
00026         throw runtime_error("Could not open file");
00027     }
00028
00029     Header header;
00030     if (header.readFromFile(file)) {
00031         cout << "Header successfully loaded from file." << endl;
00032         loadLengthIndicatedRecords();
00033     } else {
00034         loadRecords();
00035     }
00036 }
00037
00044 unordered_map<int, ZipRecord> CSVBuffer::buildPrimaryKeyIndex() const {
00045     unordered_map<int, ZipRecord> index;
00046     // Create an empty hash table to store the mapping from zipCode to ZipRecord.
00047
00048     for (const auto& record : records) {
00049         // Iterate through all ZipRecord objects stored in the 'records' vector.
00050
00051         index[record.zipCode] = record;
00052         // Insert each record into the map using its zip code as the key.
00053         // This ensures we can quickly access the full record later by zip code.
00054     }
00055
00056     return index;
00057     // Return the completed index to the caller.
00058 }
00059
00060
00065 void CSVBuffer::searchByZipCodes(const vector<int>& zipCodes) const {
00066     auto index = buildPrimaryKeyIndex();
00067     for (int zip : zipCodes) {
00068         auto it = index.find(zip);
00069         if (it != index.end()) {
00070             const auto& r = it->second;
00071             cout << "Zip Code: " << r.zipCode << ", Place Name: " << r.placeName
00072                  << ", State: " << r.state << ", County: " << r.county
00073                  << ", Latitude: " << r.latitude << ", Longitude: " << r.longitude << endl;
00074         } else {
00075             cout << "Zip Code " << zip << " not found in the file." << endl;
00076         }
00077     }
00078 }
00079
00083 void CSVBuffer::loadRecords() {
00084     string line;
00085     getline(file, line); // Skip header
00086     while (getline(file, line)) {
00087         stringstream ss(line);
00088         ZipRecord record;
00089         string temp;
00090
00091         getline(ss, temp, ',');
00092         if (temp.empty()) continue;
00093         try { record.zipCode = stoi(temp); } catch (...) { continue; }
00094         getline(ss, record.placeName, ',');
00095         getline(ss, record.state, ',');
00096         getline(ss, record.county, ',');
00097
00098         getline(ss, temp, ',');
00099         try { record.latitude = stod(temp); } catch (...) { continue; }
00100         getline(ss, temp, ',');
00101         try { record.longitude = stod(temp); } catch (...) { continue; }
00102

```

```

00103         records.push_back(record);
00104     }
00105 }
00106
00110 void CSVBuffer::loadLengthIndicatedRecords() {
00111     string line;
00112     while (getline(file, line)) {
00113         stringstream ss(line);
00114         string lenStr, data;
00115         getline(ss, lenStr, ',');
00116         getline(ss, data);
00117
00118         stringstream dataStream(data);
00119         ZipRecord record;
00120         string temp;
00121
00122         getline(dataStream, temp, ',');
00123         try { record.zipCode = stoi(temp); } catch (...) { continue; }
00124         getline(dataStream, record.placeName, ',');
00125         getline(dataStream, record.state, ',');
00126         getline(dataStream, record.county, ',');
00127
00128         getline(dataStream, temp, ',');
00129         try { record.latitude = stod(temp); } catch (...) { continue; }
00130         getline(dataStream, temp, ',');
00131         try { record.longitude = stod(temp); } catch (...) { continue; }
00132
00133         records.push_back(record);
00134     }
00135 }
00136
00142 void CSVBuffer::convertCSVToLengthIndicated(const string& inputFile, const string& outputFile) {
00143     ifstream in(inputFile);
00144     if (!in.is_open()) {
00145         cerr << "Error opening input file." << endl;
00146         return;
00147     }
00148
00149     ofstream tempOut("temp_data.tmp");
00150     if (!tempOut.is_open()) {
00151         cerr << "Error creating temporary file." << endl;
00152         return;
00153     }
00154
00155     string headerLine;
00156     getline(in, headerLine);
00157     headerLine.erase(remove(headerLine.begin(), headerLine.end(), '"'), headerLine.end());
00158     headerLine.erase(remove(headerLine.begin(), headerLine.end(), '\\n'), headerLine.end());
00159
00160     stringstream ss(headerLine);
00161     string fieldName;
00162     vector<Field> fields;
00163     int index = 0;
00164     int primaryKeyIndex = 0;
00165
00166     cout << "Parsed Fields:" << endl;
00167     while (getline(ss, fieldName, ',')) {
00168         string rawField = fieldName;
00169         fieldName.erase(remove_if(fieldName.begin(), fieldName.end(), ::isspace), fieldName.end());
00170
00171         Field field;
00172         field.name = fieldName;
00173         field.type = "string";
00174         field.format = "text/csv";
00175         field.isPrimaryKey = false;
00176         fields.push_back(field);
00177
00178         cout << "  Field " << index << ": '" << rawField << "' -> '" << fieldName << "'" << endl;
00179         ++index;
00180     }
00181
00182     if (!fields.empty()) {
00183         fields[0].isPrimaryKey = true;
00184         primaryKeyIndex = 0;
00185     }
00186
00187     int sizeInBytes = 50;
00188     for (const auto& field : fields) {
00189         sizeInBytes += field.name.length() + field.type.length() + field.format.length() + 10;
00190     }
00191
00192     int recordCount = 0;
00193     string line;
00194     while (getline(in, line)) {
00195         line.erase(remove(line.begin(), line.end(), '"'), line.end());
00196         int length = line.size();
00197         tempOut << setw(4) << setfill('0') << length << "," << line << endl;

```

```

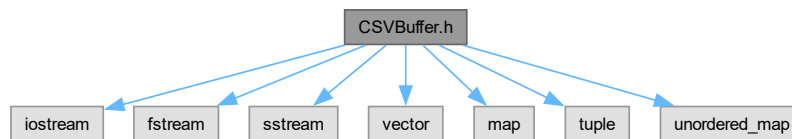
00198         ++recordCount;
00199     }
00200
00201     in.close();
00202     tempOut.close();
00203
00204     ofstream out(outputFile);
00205     if (!out.is_open()) {
00206         cerr << "Error creating output file." << endl;
00207         return;
00208     }
00209
00210     Header header("LengthIndicated", "1.0", sizeInBytes, "ASCII", "zipcode.idx",
00211         recordCount, fields, fields[primaryKeyIndex].name);
00212     header.writeToFile(out);
00213
00214     ifstream tempIn("temp_data.tmp");
00215     out << tempIn.rdbuf();
00216     tempIn.close();
00217     out.close();
00218     remove("temp_data.tmp");
00219 }
00220
00224 void CSVBuffer::generateStateTable() const {
00225     vector<ZipRecord> sortedRecords = records;
00226     string sortField;
00227     cout << "Choose sorting field (zipCode, placeName, state, latitude, longitude): ";
00228     cin >> sortField;
00229
00230     if (sortField == "zipCode") {
00231         sort(sortedRecords.begin(), sortedRecords.end(), [](const ZipRecord& a, const ZipRecord& b) {
00232             return a.zipCode < b.zipCode;
00233         });
00234     } else if (sortField == "placeName") {
00235         sort(sortedRecords.begin(), sortedRecords.end(), [](const ZipRecord& a, const ZipRecord& b) {
00236             return a.placeName < b.placeName;
00237         });
00238     } else if (sortField == "state") {
00239         sort(sortedRecords.begin(), sortedRecords.end(), [](const ZipRecord& a, const ZipRecord& b) {
00240             return a.state < b.state;
00241         });
00242     } else if (sortField == "latitude") {
00243         sort(sortedRecords.begin(), sortedRecords.end(), [](const ZipRecord& a, const ZipRecord& b) {
00244             return a.latitude < b.latitude;
00245         });
00246     } else if (sortField == "longitude") {
00247         sort(sortedRecords.begin(), sortedRecords.end(), [](const ZipRecord& a, const ZipRecord& b) {
00248             return a.longitude < b.longitude;
00249         });
00250     } else {
00251         cerr << "Invalid sorting field. Defaulting to state." << endl;
00252     }
00253
00254     map<string, tuple<ZipRecord, ZipRecord, ZipRecord, ZipRecord> stateData;
00255     for (const auto& record : sortedRecords) {
00256         auto& [east, west, north, south] = stateData[record.state];
00257         if (east.zipCode == 0 || record.longitude < east.longitude) east = record;
00258         if (west.zipCode == 0 || record.longitude > west.longitude) west = record;
00259         if (north.zipCode == 0 || record.latitude > north.latitude) north = record;
00260         if (south.zipCode == 0 || record.latitude < south.latitude) south = record;
00261     }
00262
00263     string outputFilename;
00264     cout << "Enter output filename: ";
00265     cin >> outputFilename;
00266
00267     ofstream outFile(outputFilename);
00268     if (!outFile.is_open()) {
00269         cerr << "Error: Could not open output file." << endl;
00270         return;
00271     }
00272
00273     outFile << "State, Easternmost, Westernmost, Northernmost, Southernmost" << endl;
00274     for (const auto& [state, extremes] : stateData) {
00275         const auto& [east, west, north, south] = extremes;
00276         outFile << state << ", " << east.zipCode << ", " << west.zipCode << ", "
00277             << north.zipCode << ", " << south.zipCode << endl;
00278     }
00279
00280     outFile.close();
00281     cout << "Summary written to " << outputFilename << endl;
00282 }

```

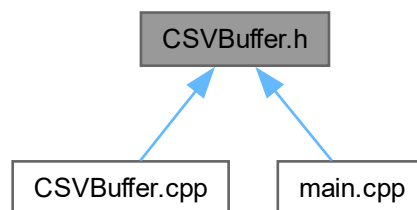
5.3 CSVBuffer.h File Reference

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
#include <map>
#include <tuple>
#include <unordered_map>
```

Include dependency graph for CSVBuffer.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct [ZipRecord](#)
Structure to store zip code information.
- class [CSVBuffer](#)
Class to handle CSV data and extract structured zip code records.

5.4 CSVBuffer.h

[Go to the documentation of this file.](#)

```
00001
00004
00005 #ifndef CSVBUFFER_H
00006 #define CSVBUFFER_H
00007
```

```

00008 #include <iostream>
00009 #include <fstream>
00010 #include <sstream>
00011 #include <vector>
00012 #include <map>
00013 #include <tuple>
00014 #include <unordered_map>
00015
00016 using namespace std;
00017
00019 struct ZipRecord {
00020     int zipCode;
00021     string placeName;
00022     string state;
00023     string county;
00024     double latitude;
00025     double longitude;
00026 };
00027
00029 class CSVBuffer {
00030 private:
00031     ifstream file;
00032     vector<ZipRecord> records;
00033
00035     void loadRecords();
00036
00038     void loadLengthIndicatedRecords();
00039
00040 public:
00043     CSVBuffer(const string& filename);
00044
00047     const vector<ZipRecord>& getRecords() const;
00048
00051     void generateStateTable() const;
00052
00056     static void convertCSVToLengthIndicated(const string& inputFile, const string& outputFile);
00057
00060     unordered_map<int, ZipRecord> buildPrimaryKeyIndex() const;
00061
00064     void searchByZipCodes(const vector<int>& zipCodes) const;
00065 };
00066
00067 #endif // CSVBUFFER_H

```

5.5 HeaderBuffer.h File Reference

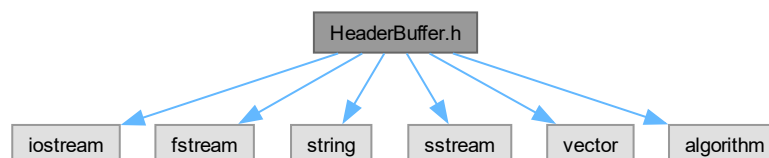
Defines a class for handling header records in data files.

```

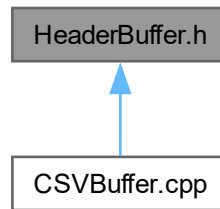
#include <iostream>
#include <fstream>
#include <string>
#include <sstream>
#include <vector>
#include <algorithm>

```

Include dependency graph for HeaderBuffer.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct [Field](#)
Structure to represent a field in the header.
- class [Header](#)
Class to manage header metadata for record-based file storage.

5.5.1 Detailed Description

Defines a class for handling header records in data files.

This class provides functionality to read and write the header record of a length-indicated CSV file, supporting structured metadata.

Definition in file [HeaderBuffer.h](#).

5.6 HeaderBuffer.h

[Go to the documentation of this file.](#)

```

00001
00005
00006 #ifndef HEADERBUFFER_H
00007 #define HEADERBUFFER_H
00008
00009 #include <iostream>
00010 #include <fstream>
00011 #include <string>
00012 #include <sstream>
00013 #include <vector>
00014 #include <algorithm>
00015
00016 using namespace std;
00017
00019 struct Field {
00020     string name;
00021     string type;
00022     string format;
00023     bool isPrimaryKey = false;
00024 };
00025
00027 class Header {
00028 public:
00029     string fileStructureType;
  
```

```

00030     string version;
00031     int sizeInBytes;
00032     string sizeFormatType;
00033     string primaryKeyFileName;
00034     int recordCount;
00035     int fieldsPerRecord;
00036     vector<Field> fields;
00037     string primaryKey;
00038
00040     Header()
00041     : fileStructureType(""), version(""), sizeInBytes(0), sizeFormatType("ASCII"),
00042       primaryKeyFileName(""), recordCount(0), fieldsPerRecord(0), primaryKey("") {}
00043
00045     Header(const string& fileStructureType, const string& version, int sizeInBytes,
00046            const string& sizeFormatType, const string& primaryKeyFileName,
00047            int recordCount, const vector<Field>& fields, const string& primaryKey)
00048     : fileStructureType(fileStructureType), version(version), sizeInBytes(sizeInBytes),
00049       sizeFormatType(sizeFormatType), primaryKeyFileName(primaryKeyFileName),
00050       recordCount(recordCount), fieldsPerRecord(fields.size()),
00051       fields(fields), primaryKey(primaryKey) {
00052         // Sanitize primary key
00053         this->primaryKey.erase(remove_if(this->primaryKey.begin(), this->primaryKey.end(),
::isspace), this->primaryKey.end());
00054     }
00055
00058     void writeToFile(ofstream& out) const {
00059         if (!out.is_open()) {
00060             cerr << "Error: Output file is not open." << endl;
00061             return;
00062         }
00063
00064         out << "HEADER,";
00065         out << fileStructureType << "," << version << "," << sizeInBytes << "," << sizeFormatType << ","
00066             << primaryKeyFileName << "," << recordCount << "," << fieldsPerRecord << "," << primaryKey <<
"\n";
00067
00068         for (const auto& field : fields) {
00069             out << field.name << "," << field.type << "," << field.format << ","
00070                 << (field.isPrimaryKey ? "true" : "false") << "\n";
00071         }
00072     }
00073
00077     bool readFromFile(ifstream& in) {
00078         if (!in.is_open()) {
00079             cerr << "Error: Input file is not open." << endl;
00080             return false;
00081         }
00082
00083         string line;
00084         if (!getline(in, line)) return false;
00085
00086         if (line.rfind("HEADER,", 0) != 0) {
00087             cerr << "Error: Missing HEADER line." << endl;
00088             return false;
00089         }
00090
00091         stringstream ss(line.substr(7)); // skip "HEADER,"
00092         string temp;
00093
00094         getline(ss, fileStructureType, ',');
00095         getline(ss, version, ',');
00096         getline(ss, temp, ','); sizeInBytes = stoi(temp);
00097         getline(ss, sizeFormatType, ',');
00098         getline(ss, primaryKeyFileName, ',');
00099         getline(ss, temp, ','); recordCount = stoi(temp);
00100         getline(ss, temp, ','); fieldsPerRecord = stoi(temp);
00101         getline(ss, primaryKey, ',');
00102         primaryKey.erase(remove_if(primaryKey.begin(), primaryKey.end(), ::isspace),
primaryKey.end());
00103
00104         fields.clear();
00105         for (int i = 0; i < fieldsPerRecord; ++i) {
00106             if (!getline(in, line)) break;
00107             stringstream fss(line);
00108             Field f;
00109             string isPK;
00110             getline(fss, f.name, ',');
00111             f.name.erase(remove_if(f.name.begin(), f.name.end(), ::isspace), f.name.end());
00112             getline(fss, f.type, ',');
00113             getline(fss, f.format, ',');
00114             getline(fss, isPK, ',');
00115             f.isPrimaryKey = (isPK == "true");
00116             fields.push_back(f);
00117         }
00118
00119         // Print full metadata
00120         cout << "Header loaded:\n";

```

```

00121     cout << "   Structure Type       : " << fileStructureType << "\n";
00122     cout << "   Version           : " << version << "\n";
00123     cout << "   Size in Bytes      : " << sizeInBytes << "\n";
00124     cout << "   Size Format Type    : " << sizeFormatType << "\n";
00125     cout << "   Primary Key File   : " << primaryKeyFileName << "\n";
00126     cout << "   Record Count      : " << recordCount << "\n";
00127     cout << "   Fields per Record : " << fieldsPerRecord << "\n";
00128     cout << "   Primary Key       : " << primaryKey << "\n";
00129     cout << "   Fields:\n";
00130
00131     for (const auto& field : fields) {
00132         cout << "       - Name: " << field.name
00133              << ", Type: " << field.type
00134              << ", Format: " << field.format
00135              << ", Primary Key: " << (field.isPrimaryKey ? "Yes" : "No") << "\n";
00136     }
00137
00138     return true;
00139 }
00140 };
00141
00142 #endif // HEADERBUFFER_H

```

5.7 main.cpp File Reference

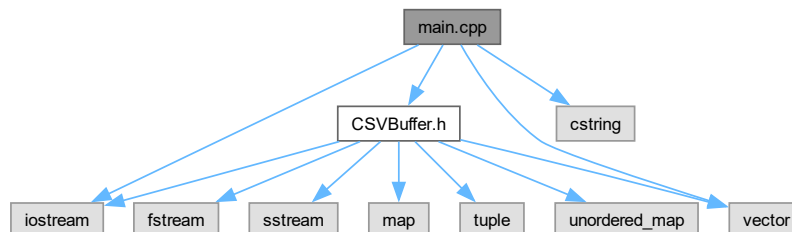
Main application to process Zip Code data from CSV or length-indicated file.

```

#include "CSVBuffer.h"
#include <iostream>
#include <vector>
#include <cstring>

```

Include dependency graph for main.cpp:



Functions

- int [main](#) (int argc, char *argv[])
Main function of the program.

5.7.1 Detailed Description

Main application to process Zip Code data from CSV or length-indicated file.

Uses [CSVBuffer](#) to generate a state-wise table of extreme Zip Codes or convert CSV to length-indicated format.

Uses [CSVBuffer](#) to generate a state-wise table of extreme Zip Codes, convert CSV files to length-indicated format, or search records by zip code using in-memory primary key indexing via command-line flags.

Definition in file [main.cpp](#).

5.7.2 Function Documentation

5.7.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Main function of the program.

Handles CSV conversion, summary generation, and zip code lookups via `-z#####` flags. If zip code flags are provided, the program loads the file, builds an index, and prints matching records. If no flags are given, it enters interactive mode.

Parameters

<i>argc</i>	Argument count from command line.
<i>argv</i>	Array of command line arguments.

Returns

int Returns 0 on success, 1 on failure.

< Stores zip codes provided via `-z#####` command-line flags.

< Filename provided to load zip code data from.

Parse command-line arguments.

Extract zip code flags and the data file to be processed.

< Extracts and stores zip code number.

< Assumes first non-flag argument is the filename.

If zip lookup flags were provided, perform index search.

< Initializes buffer and loads records.

< Searches for each zip code.

If no flags, show interactive options for CSV conversion or summary.

< User-selected operation.

Option 1 - Convert CSV to length-indicated format.

< Filenames for conversion.

< Perform conversion.

Option 2 - Generate summary of extremes per state.

< Filename for summary generation.

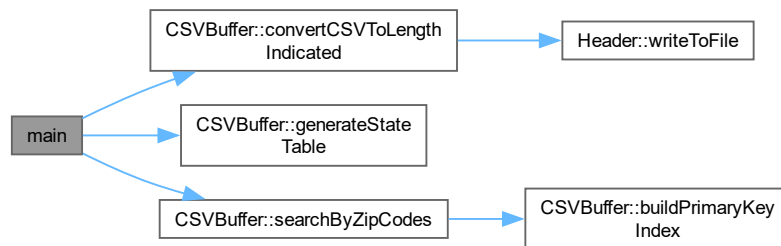
< Loads zip code records.

< Generates summary report.

< Return 0 on success.

Definition at line 23 of file [main.cpp](#).

Here is the call graph for this function:



5.8 main.cpp

[Go to the documentation of this file.](#)

```

00001
00006
00007 #include "CSVBuffer.h"
00008 #include <iostream>
00009 #include <vector>
00010 #include <cstring>
00011
00012 using namespace std;
00013
00023 int main(int argc, char* argv[]) {
00024     vector<int> zipCodes;
00025     string inputFile;
00026
00031     for (int i = 1; i < argc; ++i) {
00032         if (strcmp(argv[i], "-z", 2) == 0 || strcmp(argv[i], "-Z", 2) == 0) {
00033             try {
00034                 zipCodes.push_back(stoi(argv[i] + 2));
00035             } catch (...) {
00036                 cerr << "Invalid zip code argument: " << argv[i] << endl;
00037                 return 1;
00038             }
00039         } else {
00040             inputFile = argv[i];
00041         }
00042     }
00043
00047     if (!zipCodes.empty()) {
00048         if (inputFile.empty()) {
00049             cerr << "Error: Please specify the filename before zip code flags." << endl;
00050             return 1;
00051         }
00052         try {
00053             CSVBuffer buffer(inputFile);
00054             buffer.searchByZipCodes(zipCodes);
00055         } catch (const exception& e) {
00056             cerr << "Error: " << e.what() << endl;
00057             return 1;
00058         }
00059         return 0;
00060     }
00061
00065     cout << "Choose operation:\n1. Convert CSV to length-indicated file\n2. Process file and generate
summary\nEnter choice (1 or 2): ";
00066     int choice;
00067     cin >> choice;

```

```
00068
00072     if (choice == 1) {
00073         string inputFile, outputFile;
00074         cout << "Enter input CSV filename: ";
00075         cin >> inputFile;
00076         cout << "Enter output filename for length-indicated file: ";
00077         cin >> outputFile;
00078         CSVBuffer::convertCSVToLengthIndicated(inputFile, outputFile);
00079         cout << "Conversion complete.\n";
00080     }
00084     else if (choice == 2) {
00085         string inputFile;
00086         cout << "Enter filename to process: ";
00087         cin >> inputFile;
00088         try {
00089             CSVBuffer buffer(inputFile);
00090             buffer.generateStateTable();
00091         } catch (const exception& e) {
00092             cerr << "Error: " << e.what() << endl;
00093             return 1;
00094         }
00095     } else {
00096         cout << "Invalid choice. Exiting.\n";
00097         return 1;
00098     }
00099
00100     return 0;
00101 }
00102
```

5.9 README.md File Reference

Index

- buildPrimaryKeyIndex
 - CSVBuffer, [9](#)
- convertCSVToLengthIndicated
 - CSVBuffer, [9](#)
- county
 - ZipRecord, [19](#)
- CSVBuffer, [7](#)
 - buildPrimaryKeyIndex, [9](#)
 - convertCSVToLengthIndicated, [9](#)
 - CSVBuffer, [8](#)
 - file, [13](#)
 - generateStateTable, [10](#)
 - getRecords, [10](#)
 - loadLengthIndicatedRecords, [11](#)
 - loadRecords, [11](#)
 - records, [13](#)
 - searchByZipCodes, [11](#)
- CSVBuffer.cpp, [21](#)
- CSVBuffer.h, [25](#)
- Field, [14](#)
 - format, [14](#)
 - isPrimaryKey, [14](#)
 - name, [14](#)
 - type, [14](#)
- fields
 - Header, [17](#)
- fieldsPerRecord
 - Header, [17](#)
- file
 - CSVBuffer, [13](#)
- fileStructureType
 - Header, [17](#)
- format
 - Field, [14](#)
- generateStateTable
 - CSVBuffer, [10](#)
- getRecords
 - CSVBuffer, [10](#)
- Header, [15](#)
 - fields, [17](#)
 - fieldsPerRecord, [17](#)
 - fileStructureType, [17](#)
 - Header, [16](#)
 - primaryKey, [17](#)
 - primaryKeyFileName, [17](#)
 - readFromFile, [16](#)
- recordCount, [18](#)
- sizeFormatType, [18](#)
- sizeInBytes, [18](#)
- version, [18](#)
- writeToFile, [16](#)
- HeaderBuffer.h, [26](#)
- isPrimaryKey
 - Field, [14](#)
- latitude
 - ZipRecord, [19](#)
- loadLengthIndicatedRecords
 - CSVBuffer, [11](#)
- loadRecords
 - CSVBuffer, [11](#)
- longitude
 - ZipRecord, [19](#)
- main
 - main.cpp, [30](#)
- main.cpp, [29](#)
 - main, [30](#)
- name
 - Field, [14](#)
- placeName
 - ZipRecord, [19](#)
- primaryKey
 - Header, [17](#)
- primaryKeyFileName
 - Header, [17](#)
- readFromFile
 - Header, [16](#)
- README.md, [32](#)
- recordCount
 - Header, [18](#)
- records
 - CSVBuffer, [13](#)
- searchByZipCodes
 - CSVBuffer, [11](#)
- sizeFormatType
 - Header, [18](#)
- sizeInBytes
 - Header, [18](#)
- state
 - ZipRecord, [20](#)

type

Field, [14](#)

version

Header, [18](#)

writeToFile

Header, [16](#)

zipCode

ZipRecord, [20](#)

ZipCode2.0, [1](#)

ZipRecord, [18](#)

county, [19](#)

latitude, [19](#)

longitude, [19](#)

placeName, [19](#)

state, [20](#)

zipCode, [20](#)